Chapter 10

ShortLab

In this assignment, you will implement a solution to the *unweighted* variety of the *all single-source shortest paths* problem. Given a graph and a source, you will need to compute every shortest path to every other vertex in the graph. As an example application, these solutions can then be used to find the shortest path(s) of synonyms between two words.

Then we will use unweighted and weighted shortest paths to solve algorithmic written problems.

1 Files

After downloading the assignment tarball from Ed, extract the files by running:

```
tar -xvf ShortLab-handout.tgz
```

from a terminal window.

Some of the files worth looking at are listed below. The files denoted by * will be handed in by the submission script.

1. `Makefile`
2. * `MkUnweightedASP.sml`
3. `Sandbox.sml`
4. `Tests.sml`

Additionally, you should create a file called `written.pdf` which contains the answers to the written parts of the assignment.

2 Submission

To submit the code portion of the assignment, run `make package` and upload the resulting `handin.zip` to the Gradescope code folder. You should ensure your code compiles by checking for a score of 0 under the `Compilation` section on Gradescope. Note that Gradescope can take a few minutes to compile code submissions.

Submit the written portion by uploading your `written.pdf` to the Gradescope written folder.

3 Unweighted Shortest Paths

Suppose you are interested in finding how closely related two words are. In order to find the similarity between two words, you decide to find the shortest path of synonyms relating the first word to the second word. In particular, you are only interested in finding the relationship between your favorite word and all other words so you preprocess all possible shortest paths from you favorite word to the remaining words you are interested in.

3.1 Implementation

Note. For cost bounds, assume

- the `ArraySequence` implementation of `SEQUENCE`, and
- the `MkTreapTable` implementation of `TABLE`.

The graphs in this section are *directed*, *simple*, and *unweighted*.

Task (1). (65 points) In `MkUnweightedASP.sml`, implement the following functions. Note that you will first need to decide on a representation for the types `graph` and `asp`. Choose these carefully so that you can meet the given cost bounds.

```
val makeGraph : edge Seq.t -> graph
```

Given a sequence of directed edges E, `makeGraph E` should produce the graph induced by E. Your implementation should have $O(|E| \log |E|)$ work and $O(\log^2 |E|)$ span.

```
val makeASP : graph -> vertex -> asp
```

Given a graph G, `makeASP G s` should return an `asp` containing information about all shortest paths in G beginning at the source s. Your implementation should have $O(|E| \log |V|)$ work and $O(D \log^2 |V|)$ span, where D is the diameter of the graph. You may assume that all vertices are reachable from the source.

```
val report : asp -> vertex -> vertex Seq.t Seq.t
```

Given an `asp` A which was constructed for the graph (V, E) with source s, `report A t` should return all shortest paths from s to t. Each path is a sequence of vertices beginning at s and ending at t. Your output doesn't have to be ordered in any particular way. Your implementation should have $O(k\ell \log |V|)$ work and span where

- k is the number of shortest paths between s and t, and
- ℓ is the length of the shortest path between s and t.

4 Testing

It is very important that you thoroughly test your code before you submit. After the deadline, we will grade your code with a private set of test cases.

We provide three ways to test your code with SML/NJ.

1. In `Sandbox.sml`, write whatever testing code you'd like. You can then access the sandbox at the REPL:

```
- CM.make "sandbox.cm";
...
- open Sandbox;
```

2. In `Tests.sml`, add test cases according to the instructions given. Then run the autograder:

```
- CM.make "autograder.cm";
...
- Autograder.run ();
```

3. You can load `thesaurus.cm` and print the shortest paths from one word to another, as follows:

```
- CM.make "thesaurus.cm"; open Thesaurus;
...
- val goodPaths = unweightedQuery "good";
val goodPaths = fn : string -> unit
- goodPaths "evil";
UNWEIGHTED PATHS FROM good TO evil:
good, value, appreciate, prize, prey, spoil, damage, harm, evil
good, value, rate, tempo, beat, throb, ache, hurt, evil
good, value, appreciate, prize, prey, spoil, damage, hurt, evil
good, right, accurate, exact, extort, wring, pain, hurt, evil
good, value, treasure, wealth, fortune, chance, accident, mishap, evil
good, worth, merit, reward, tip, upset, bother, nuisance, evil
val it = () : unit
```

   The thesaurus does not adhere to the assumption that all vertices are reachable from the source but any tests that we run will satisfy this requirement.

5 Algorithm Design Questions

For all of the questions in this section, you should describe your algorithm at a high level, in English as opposed to pseudocode. Feel free to cite results from the lecture notes. Your answers should be precise, specific, and brief.

5.1 Mongolian Puzzle

The infamous Mongolian puzzle-warrior Vidrach Itky Leda invented the following puzzle in the year 1473. The puzzle consists of an $n \times n$ grid of squares, where each square is labeled with a positive integer. In addition, there are two tokens, one red and the other blue, which must lie on distinct squares of the grid. The red token starts in the top left corner, and the blue token starts in the bottom right corner. The goal of the puzzle is to

move the red token to the bottom right and the blue token to the top left; that is, to swap the positions of the tokens.

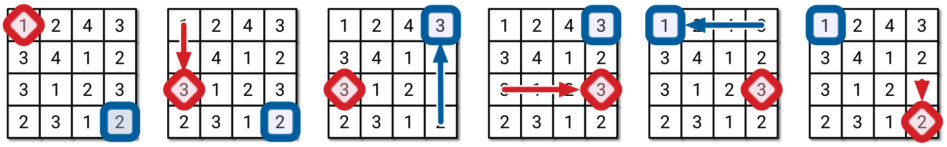
In a single turn, a token may be moved up, down, left, or right by a distance exactly equal to the positive integer on the square on which the other token lies. However, you may not move either token off the grid, and at the end of a move the two tokens cannot lie on the same square (but a move of one token can "jump over" the other). For the initial board in the example below, since the blue token is on a square labeled 2, then you may move the red token exactly 2 steps down or 2 steps right because moving 2 steps up or left would go off the board.

The red token must be moved first, and the token being moved must alternate on each turn. This means that if red moves in one turn, then the blue token must move in the next turn and vice-versa.

Your task is to build an algorithm that returns the minimum number of moves needed to solve a given Vidrach Itky Leda puzzle.

Note. This problem is courtesy of Jeff Erickson, whose algorithms textbook you can find here: http://algorithms.wtf . Feel free to check out this book: there are a lot of great algorithm design problems! However, Jeff is careful to ensure that solutions to his problems are not available.

Example 10.1. This puzzle can be solved with the following 5 moves.



Task (2)**.** (20 points) Describe an algorithm that returns the minimum number of moves required to solve a given Vidrach Itky Leda puzzle or reports that the puzzle has no solutions. Your algorithm should have $O(n^4)$ work where the input grid is of size $n \times n$. Briefly justify the correctness of your algorithm and that your algorithm is in cost bounds.

Hint. As as guide, our reference solution is around 10 sentences long, including cost justification. It's ok if your solution is longer, but we'd appreciate concise solutions.

5.2 Building Roads

Construction Mayhem University would like to build a new road between buildings. CMU currently has n buildings connected by m bidirectional roads. Each road connects two distinct buildings and no two roads connect the same pair of buildings. It is possible to get from any building to any other building by these roads. The distance between two buildings is equal to the minimum possible number of roads on a path between them.

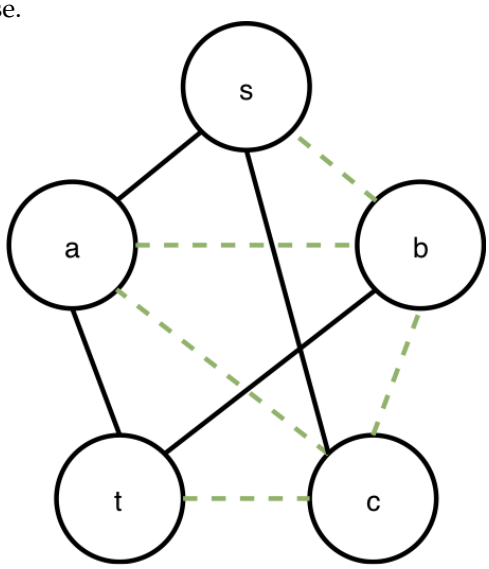
In order to improve the transportation system, President Nearnam would like to build a new road. He would like to poll the students on which two currently unconnected buildings to connect.

However, the students would like to mess with Professor Umut A. Afar. They want to ensure that the road added does not shorten Umut's commute from building s to building t. Assume that Umut takes the path from s to t that uses the minimum possible number of roads.

Task (3)**.** (25 points) Describe an algorithm which computes the number of pairs of buildings that are not connected by a road, such that if the new road between these two buildings is built the distance between s and t won't decrease. Your algorithm should take $O(n^2)$ work and span, assuming the buildings are enumerated. Briefly justify the correctness of your algorithm and that your algorithm is in cost bounds.

Hint. As as guide, our reference solution is around 6 sentences long, including cost justification. It's ok if your solution is longer, but we'd appreciate concise solutions.

Example 10.2. The black edges represent roads which already exist, and so the shortest distance from s to t is 2. There are 5 roads which could be added that do not decrease this distance, represented by the green dashed lines, so your algorithm should return 5 in this case.



5.3 Budget Cuts

There are $n \geq 1$ cities including a designated capital city c. There are m bidirectional bus routes between cities. There are also k bidirectional helicopter routes which only fly between c and other cities. Each of the bus and helicopter routes has a positive duration (weight) associated with it.

You may assume that there is at least one path from the capital to every city, and that the graph has no self-loops. However, there may be multiple bus routes and/or helicopter routes (edges) between the same pair of cities.

Due to recent tent-related expenditures, President Nearnam needs a way to cut costs. He has decided to cancel the maximum number of helicopter routes such that the shortest duration between every city to the capital does not change.

Task (4)**.** (20 points) Describe an algorithm which computes the maximum number of helicopter routes that can be cancelled such that the total duration of the shortest paths from every city to the capital stays the same. Your algorithm should take $O(m \log n + k \log n)$ work and span. Briefly justify the correctness of your algorithm and that your algorithm is in cost bounds.

Hint. As as guide, our reference solution is around 10 sentences long, including cost justification. It's ok if your solution is longer, but we'd appreciate concise solutions.